

ALGORITMI PARALELI ȘI DISTRIBUIȚI

Curs

Facultatea de Electronică, Comunicații și Calculatoare
Universitatea Națională de Știință și Tehnologie Politehnica
București – Centrul universitar Pitești

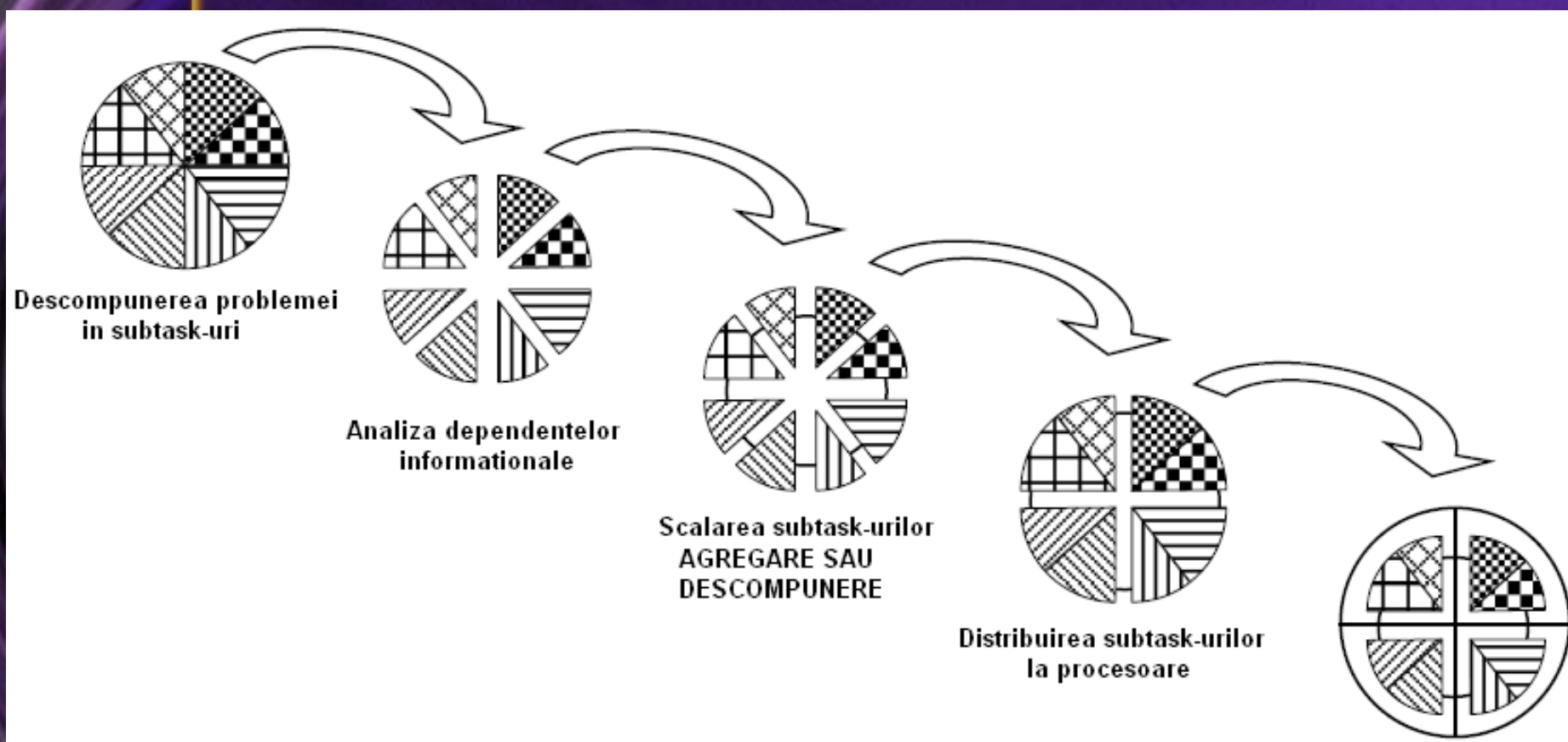
Algoritmi paraleli și distribuiți

Exemple de algoritmi paraleli și distribuiți.

Proiectare și analiza.

Algoritmi paraleli de înmulțire a matricelor cu vectori

Etape de realizare a algoritmilor paraleli (**vezi și curs 3**)



Foster, I. (1995). Designing and Building Parallel Programs: Concepts and Tools for Software Engineering. Reading, MA: Addison-Wesley.

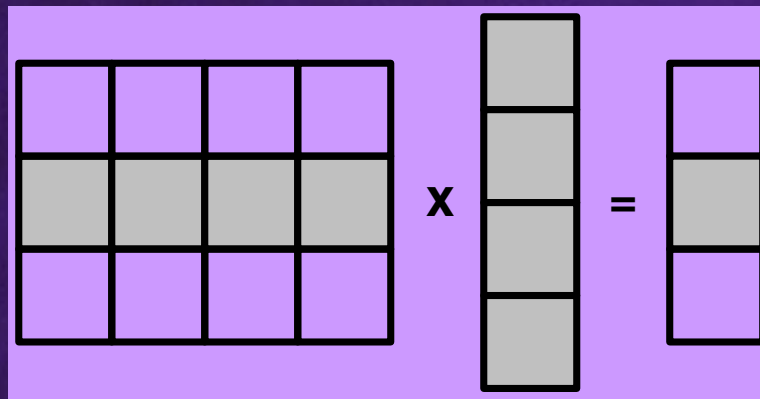
Algoritmi paraleli de înmulțire a matricelor cu vectori

**Operațiile cu matrice sunt
intens utilizate în diverse
aplicații științifice și
inginerești.**

Algoritmi paraleli de înmulțire a matricelor cu vectori

Problema înmulțirii matricelor cu vectori:

$$c = A \cdot b$$



$$c_i = (a_i, b) = \sum_{j=1}^n a_{ij} b_j, \quad 1 \leq i \leq m$$

Algoritmi paraleli de inmultire a matricelor cu vectori

$$\begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_{m-1} \end{pmatrix} = \begin{pmatrix} a_{0,0} & a_{0,1} & \dots & a_{0,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m-1,0} & a_{m-1,1} & \dots & a_{m-1,n-1} \end{pmatrix} \begin{pmatrix} b_0 \\ b_1 \\ \vdots \\ b_{n-1} \end{pmatrix}$$

Algoritmi paraleli de inmultire a matricelor cu vectori

Analiza numarului de operatii:

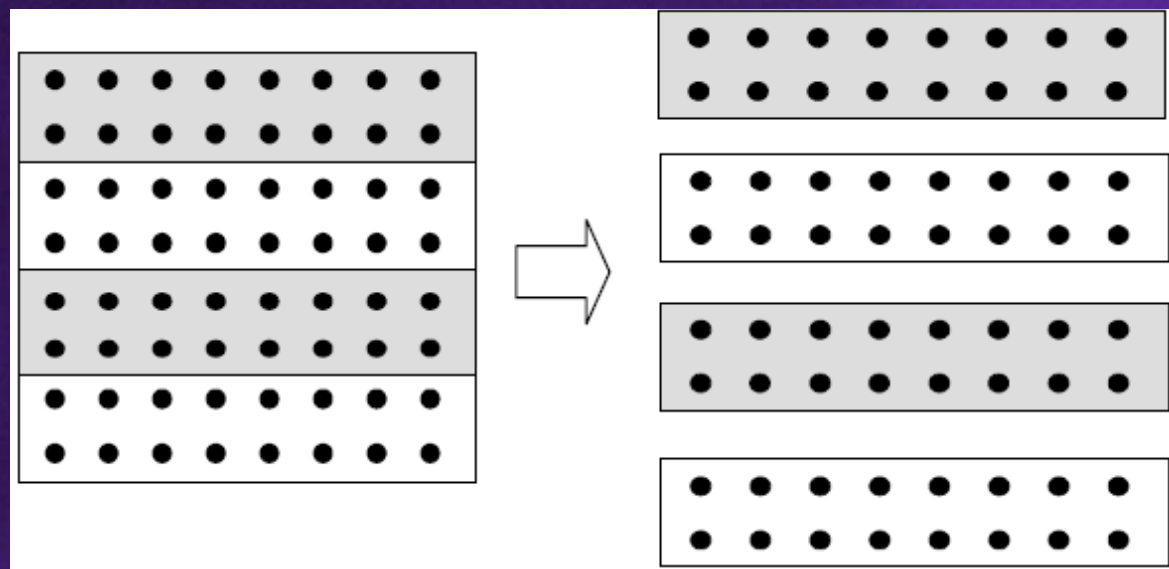
$$N = Dim \bullet (2 \bullet Dim - 1)$$

Timp teoretic de calcul:

$$T_1 = N \bullet \tau = Dim \bullet (2 \bullet Dim - 1) \bullet \tau$$

Scheme de descompunere paralelă (1):

a) linii



$$A = (A_0, A_1, \dots, A_{p-1})^T,$$

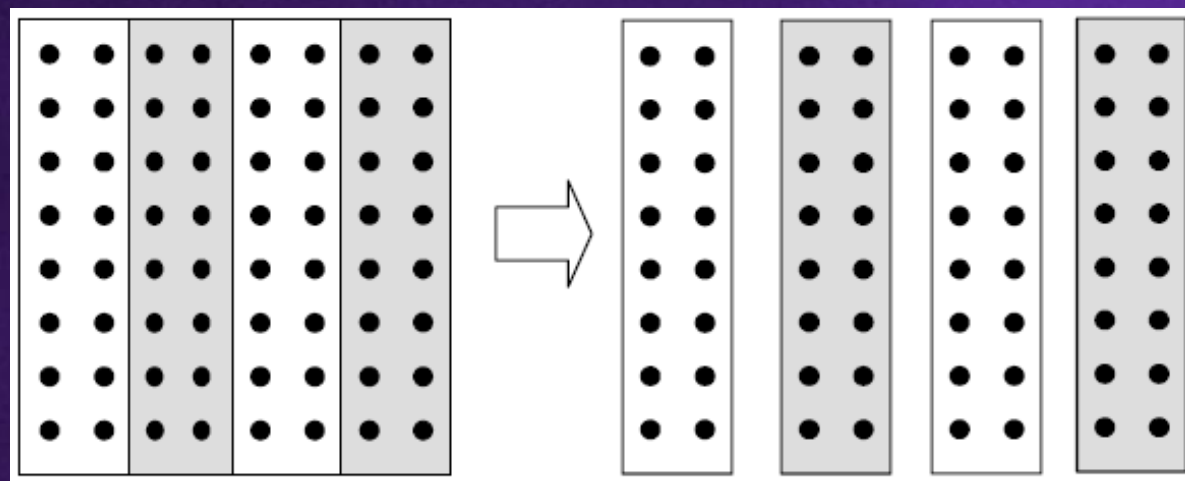
$$A_i = (a_{i_0}, a_{i_1}, \dots, a_{i_{k-1}}),$$

$$i_j = ik + j, 0 \leq j < k, k = m / p$$

Scheme de descompunere paralelă (2):

Descompunere
(partitionare)
sub forma de
blocuri:

b) coloane



$$A = (A_0, A_1, \dots, A_{p-1}),$$

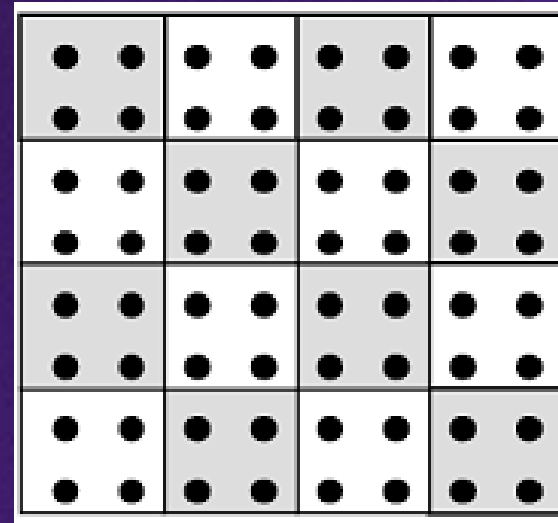
$$A_i = (\alpha_{i_0}, \alpha_{i_1}, \dots, \alpha_{i_{k-1}}),$$

$$i_j = il + j, 0 \leq j < l, l = n / p$$

Scheme de descompunere paralelă (3):

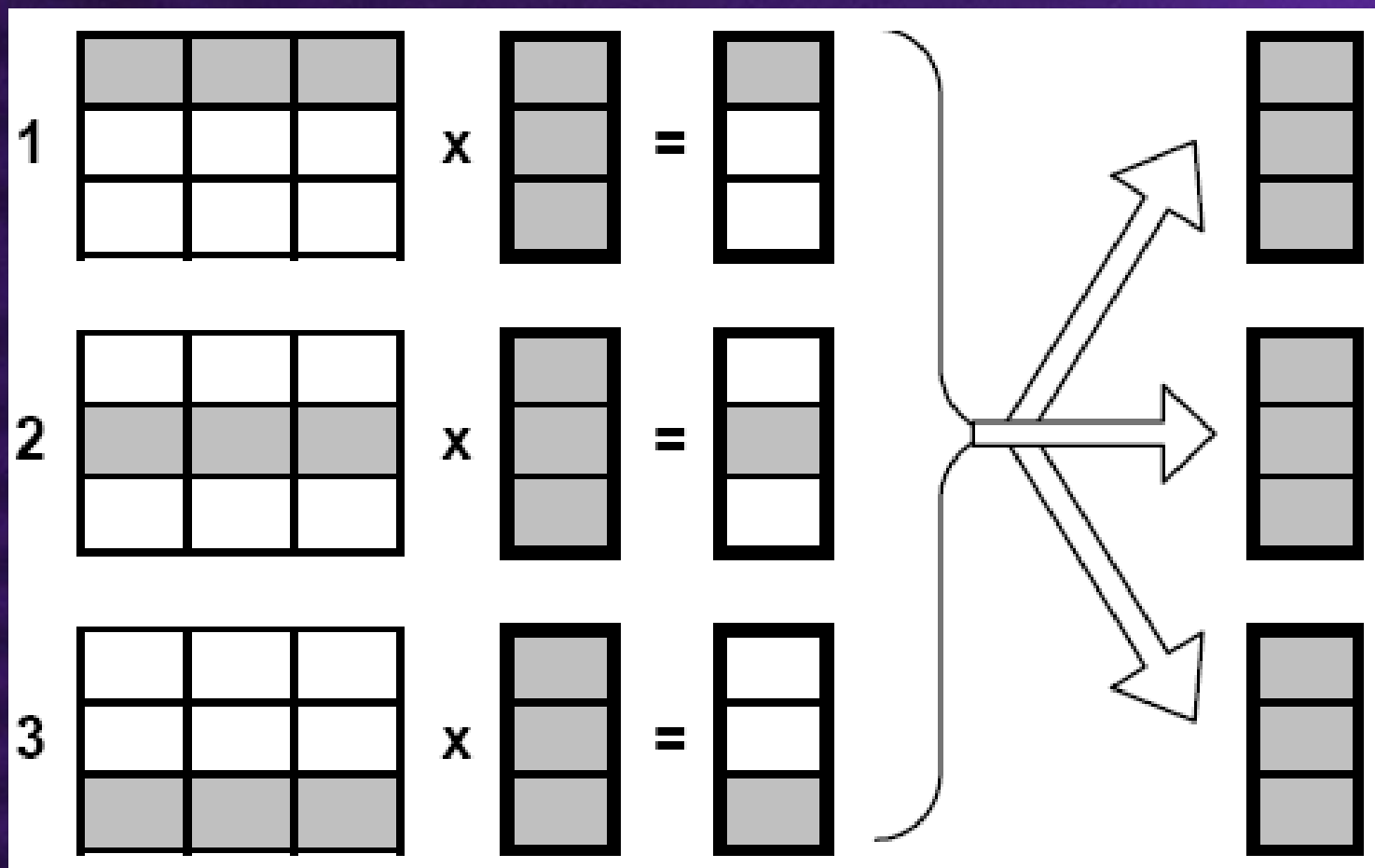
Descompunere
(partitionare) sub
forma de blocuri:

c) rectangulare
sau patrate



$$A = \begin{pmatrix} A_{00} & A_{02} & \dots & A_{0q-1} \\ & & \dots & \\ A_{s-11} & A_{s-12} & \dots & A_{s-1q-1} \end{pmatrix}$$

Analiza dependentelor informationale pentru algoritmul 1 - linii



Scalarea subtask-urilor pentru algoritmul 1 - linii

Agregare

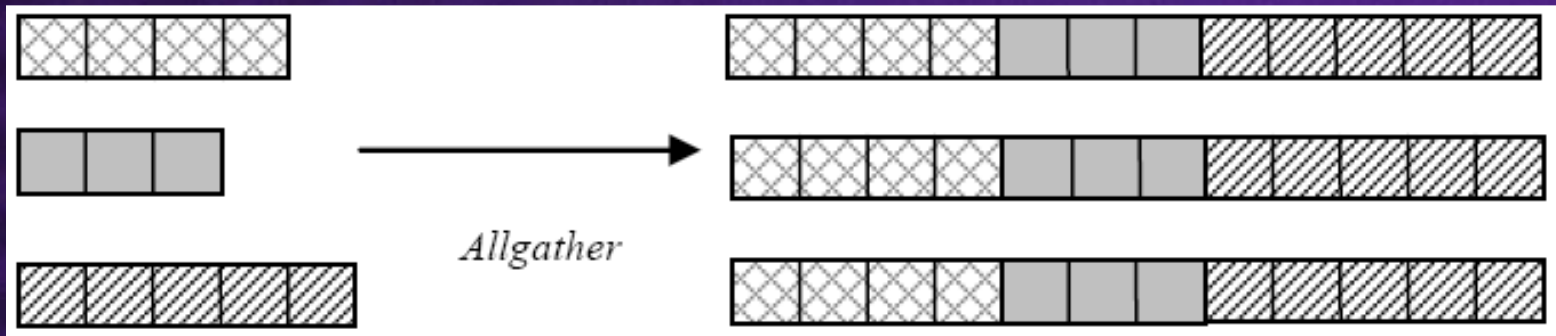
sau

Descompunere

???

Distribuirea subtask-urilor între procesoare

1. *MPI_Bcast* – pentru ???,
2. *MPI_Scatter* – pentru ???,
3. *MPI_Allgather* – pentru ???.



Analiza performantelor algoritmului

1 - linii

Accelerarea paralelă (parallel speedup – S_p).

$$1 \leq S_p = \frac{T_s}{T_p} = \frac{p}{1 + f(p-1)} \leq p$$

Legea lui Amdahl (1967)

Considerand matrice patrata ($m=n$) avem:

$$S_p = \frac{n^2}{n^2 / p} = p$$

$$E_p = \frac{n^2}{p \cdot (n^2 / p)} = 1$$

Analiza performantelor algoritmului

1 - linii

Analiza eficienței (Estimări detaliate).

Timpul de execuție PC:

$$T_p(\text{calc}) = \lceil n/p \rceil \cdot (2n - 1) \cdot \tau$$

Timpul pentru operațiile All-gather se poate determina utilizând modelul Hockney:

$$T_p(\text{comm}) = \sum_{i=1}^{\lceil \log_2 p \rceil} (\alpha + 2^{i-1} w \lceil n/p \rceil / \beta) = \alpha \lceil \log_2 p \rceil + w \lceil n/p \rceil (2^{\lceil \log_2 p \rceil} - 1) / \beta$$

Timpul total pentru execuția paralelă este:

$$T_p = (n/p) \cdot (2n - 1) \cdot \tau + \alpha \cdot \log_2 p + w(n/p)(p - 1) / \beta$$

Descrierea codului algoritmului

1 - linii (secvente de cod)

Functia: **MAIN**

1. Realizeaza logica de control
2. Apeleaza subprogramele necesare.

Functia: **InitializareProces**

1. Stabileste dimensiunea obiectelor
2. Defineste datele initiale

Functia: **DistributieDate**

1. Trimite vectorul
2. Distribuie randuri din matrice

Descrierea codului algoritmului

1 - linii (secvente de cod)

Functia: `CalculareParalelaRezultat`

1. Realizeaza inmultirea randurilor matricei cu vectorul
2. Construiește blocuri din vectorul rezultat (final).

Functia: `ReplicareRezultate`

1. Uneste blocurile de date ale vectorului rezultat
2. Trimite vectorul rezultat la toate procesoarele ce compun clusterul.

Testare & Evaluare

